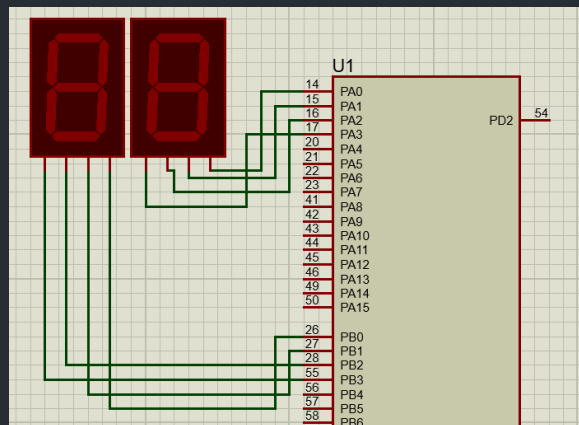


سوالات تحلیلی

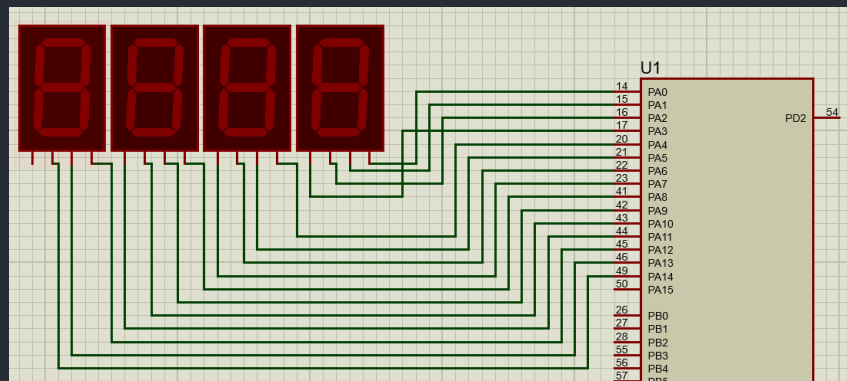
سوال 1: چند روش برای راه اندازی چهار عدد نمایشگر 7seg به طور همزمان با یک میکروکنترلر وجود دارد؟ با رسم اتصالات شرح دهید.

در ابتدا به این اشاره کنیم 7seg دارای دو تایپ است، یکی آنکه دارای 4 بیت ورودی بوده و با گرفتن 4 بیت BCD، رقم را نشان می‌دهد. یک نوع دیگر آن که در صورت آزمایش آمده است دارای 8 ورودی است، 7 تای آن برای فعال کردن هر segment است و دیگر برای وصل کردن آن به supply یا ground استفاده می‌شود (که این خود دارای دو تایپ کاتد مشترک و آند مشترک است). راه های پیاده سازی 4 نمایشگر به این شکل است:

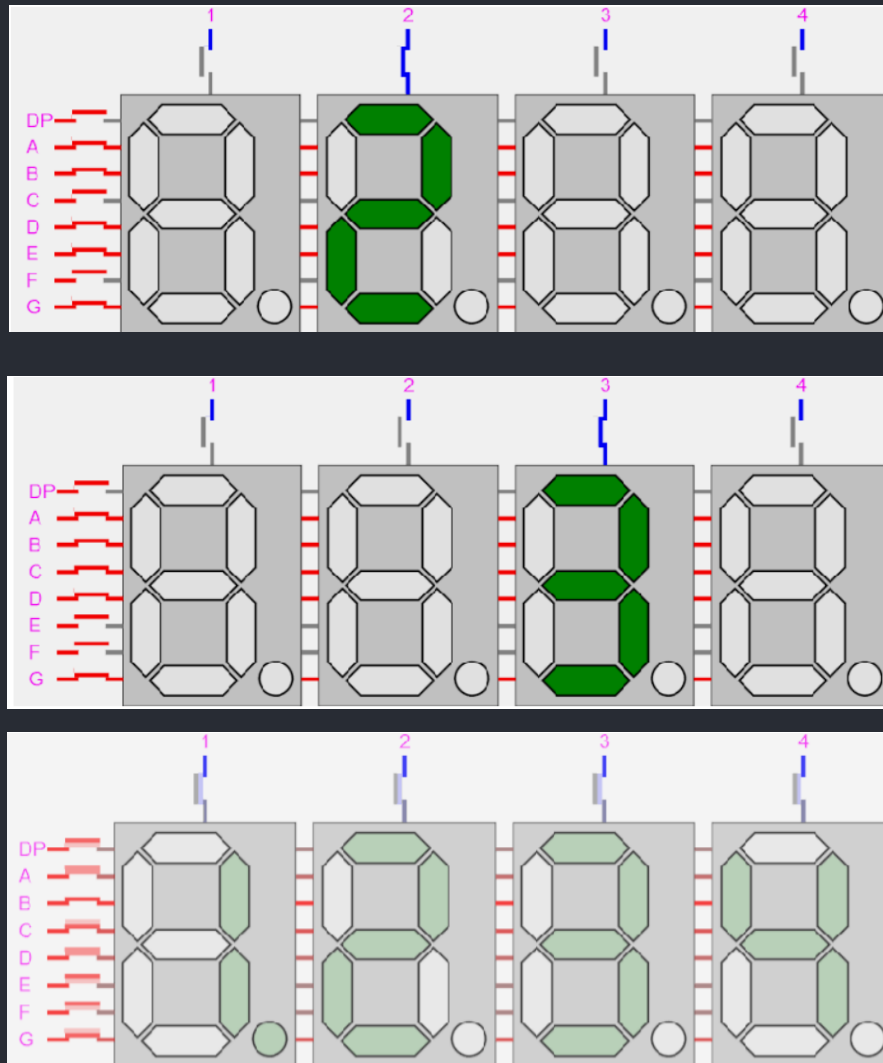
1. هر نمایشگر را به پورت های جداگانه متصل کنیم. از معایب این روش اشغال شدن پورت های زیاد و redundancy است. یک چیزی مثل این:



2. هر 4 تا نمایشگر را به یک پورت وصل کنیم و از تمام ظرفیت آن پورت استفاده کنیم (اگر 7 سگمنت ما 8 ورودی باشد هر دو تا را به یک پورت وصل می‌کنیم):



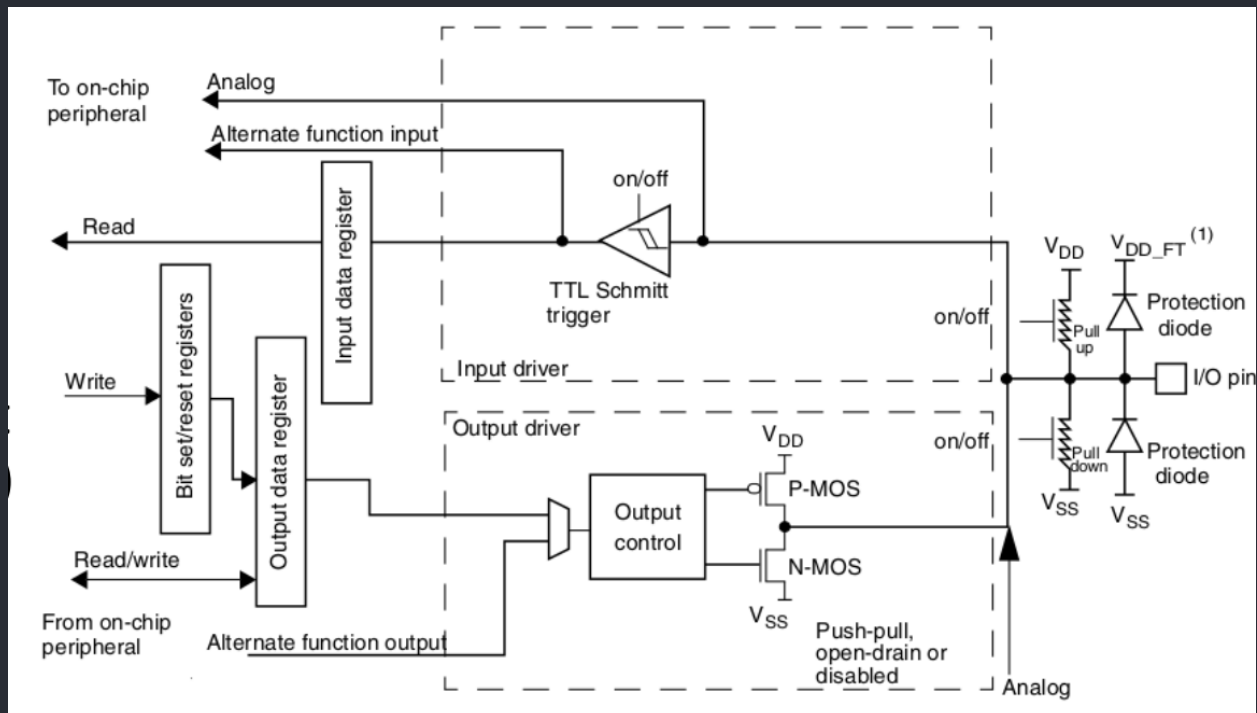
3. یک روش دیگر مالتی پلکس کردن یک به یک و مرتب 7seg ها به صورت بی وقفه می‌باشد. به این صورت که داخل یک حلقه یک به یک نمایشگر ها روشن می‌شوند و مقدار مربوط به هریک در آن لحظه assign می‌شود:



از آنجایی که چشم انسان دارای یک محدودیت برای دیدن فریم هاست، در هر لحظه چشم این 4 نمایشگر را با مقادیر مربوط به آن ها روشن می‌بیند. حداکثر بیت لازم برای اینکار 12 بیت است. (با یک پورت می‌توان آن را هندل کرد).

4. همانند قسمت قبل اما با استفاده از شیفت رجیستر و ذخیره مقادیر هر نمایشگر در رجیستر ها.

سوال ۲: بلوک دیاگرام GPIO در STM32 از چه بخشهایی تشکیل شده است؟ نام برده و وظیفه هر بخش را توضیح دهید.



- **Protection diodes:**

این دیود ها برای جلوگیری از احتمال آسیب به مدار داخلی GPIO در زمان هایی که ولتاژ بالاتر از سطح انتظار به آن وارد می شود، تعبیه شده است.

- **Pull-up and pull-down resistors:**

این رزیستور ها وضعیت دیفالت پین را وقتی که در حالت input قرار دارد، تنظیم می کند (در سوال بعد بیشتر توضیح داده شده است). رزیستور pull-up، پین را به منبع ولتاژ و رزیستور pull-down، پین را به زمین متصل می کند.

- **Input driver:**

این درایور، مقادیر دیجیتال (صفر و یک) را از سیگنال های خارجی روی پین می خواند و آن را به مدار داخلی GPIO ارسال می کند. حساسیت این درایور به ولتاژ و مقادیر منطقی آن قابل تنظیم است.

- **Output driver:**

این درایور، مقادیر دیجیتال (صفر و یک) را به مدار بیرونی پین می دهد و خروجی را کنترل می کند. همانطور که در سوال بعد توضیح داده می شود این درایور در چندین مود کار خواهد کرد.

سوال ۳: مدهای کاری GPIO در STM32F401 را با ذکر رجیسترهای مربوط به این مدها توضیح دهید.

مود کاری GPIO با رجیستر زیر مشخص می‌شود:

- GPIOx_MODER (to select the mode (Input, Output, Alternate, Analog))

مود های کاری آن به شرح زیر می‌باشد:

- **Input mode:**

در این مود پین مورد نظر به عنوان ورودی عمل می‌کند. که دارای سه حالت است:

1. Input with internal pull-up
2. Input with internal pull-down
3. Floating input

رجیستر های مربوط به این مود به همراه توضیحات‌شان:

- **GPIOx_PUPDR** (The pull-up or pull-down resistors is set by this register)
- **GPIOx_IDR** (For Inputting Data)

- **Output mode:**

در این مود پین مورد نظر به عنوان خروجی عمل می‌کند.

رجیستر های مربوط به این مود به همراه توضیحات‌شان:

- **GPIOx_OTYPER** (The control of the two transistors(NMOS and PMOS) is done through this register)
 - **GPIOx_ODR:**
 - if 1: activates the PMOS transistor to force the I/O pin to V_{DD}
 - if 0: activates the NMOS transistor to force the I/O pin to ground
- **GPIOx_PUPDR** (The internal pull-up and pull-down resistors are activated by this register)
- **GPIOx_ODR** (For outputting data)

- **Alternate Functions mode:**

در این مود کاربر می‌تواند علاوه بر عملکرد IO، عملکرد ها و فانکشن های دیگری را برای پین ست کند. (مثل communication interfaces (SPI, UART, I2C, USB, CAN, LCD and others), timers, debug interface و غیره). این عملکرد ها با این دو رجیستر ست می‌شوند:

- **GPIOx_AFRL** (for pin 0 to 7)
- **GPIOx_AFRH** (for pin 8 to 15)

دیگر رجیستر ها:

- **GPIOx_PUPDR** (The pull-up or pull-down resistors is set by this register)
- **Analog mode:**

بعضی از پین های GPIO می‌تواند در این مود قرار بگیرند. در این مود این پین ها به حالت آنالوگ می‌روند که برای استفاده از ADC, DAC, OPAMP, and COMP internal peripherals کاربرد دارد.

رجیستر های مربوط به این مود به همراه توضیحاتشان:

- **GPIOx_ASCR** (to select the required function ADC, DAC, OPAMP, or COMP)

سوال ۴: با مراجعه به مستندات برد Nucleo STM32F401 بگویید کدام یک از پینهای GPIO به اجزای روی برد متصل هستند و برای کاربرد دلخواه قابل بکارگیری نیستند؟

Push-buttons

B1 USER: the user button is connected to the I/O PC13 (pin 2) of the STM32 microcontroller.

B2 RESET: this push-button is connected to NRST, and is used to RESET the STM32 microcontroller.

User LD2: the green LED is a user LED connected to Arduino signal D13 corresponding to STM32 I/O PA5 (pin 21) or PB13 (pin 34) depending on the STM32 target. Refer to [Table 11](#) to [Table 23](#) when:

- the I/O is HIGH value, the LED is on
- the I/O is LOW, the LED is off

Table 14. Arduino connectors on NUCLEO-F303RE

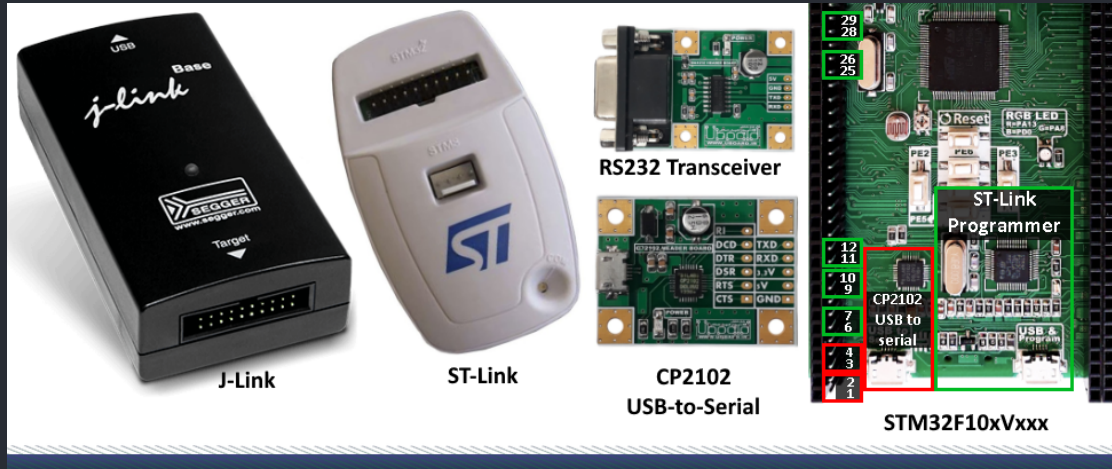
Connector	Pin	Pin name	STM32 pin	Function
Left connectors				
CN6 power	1	NC	-	-
	2	IOREF	-	3.3V Ref
	3	RESET	NRST	RESET
	4	+3.3V	-	3.3V input/output
	5	+5V	-	5V output
	6	GND	-	ground
	7	GND	-	ground
	8	VIN	-	Power input
CN8 analog	1	A0	PA0	ADC1_IN1
	2	A1	PA1	ADC1_IN2
	3	A2	PA4	ADC2_IN1
	4	A3	PB0	ADC3_IN12
	5	A4	PC1 or PB9 ⁽¹⁾	ADC12_IN7 (PC1) or I2C1_SDA (PB9)
	6	A5	PC0 or PB8 ⁽¹⁾	ADC12_IN6 (PC0) or I2C1_SCL (PB8)
Right connectors				
CN5 digital	10	D15	PB8	I2C1_SCL
	9	D14	PB9	I2C1_SDA
	8	AREF	-	AVDD
	7	GND	-	ground
	6	D13	PA5	SPI1_SCK
	5	D12	PA6	SPI1_MISO
	4	D11	PA7	TIM17_CH1 or SPI1_MOSI
	3	D10	PB6	TIM4_CH1 or SPI1_CS
	2	D9	PC7	TIM3_CH2
	1	D8	PA9	-

Table 14. Arduino connectors on NUCLEO-F303RE (continued)

Connector	Pin	Pin name	STM32 pin	Function
CN9 digital	8	D7	PA8	-
	7	D6	PB10	TIM2_CH3
	6	D5	PB4	TIM3_CH1
	5	D4	PB5	-
	4	D3	PB3	TIM2_CH2
	3	D2	PA10	-
	2	D1	PA2	USART2_TX

1. Refer to [Table 10: Solder bridges](#) for details.

سوال ۵: هنگام پروگرام کردن میکروکنترلر STM32F401 چه پایه هایی درگیر بوده و در چه مدی باید قرار گیرند؟



چند روش برای پروگرام کردن این میکروکنترلر وجود دارد:

1. JTAG method:

برای پروگرام کردن در این روش باید پین های TDI, TMS, TCK, TDO, RST, VDD and GND پروگرامر به پین های AF0, AF0, AF0, AF0, AF0, AF12 and AF0 به ترتیب وصل شوند. این پین ها باید به حالت Alternate Function ست شوند.

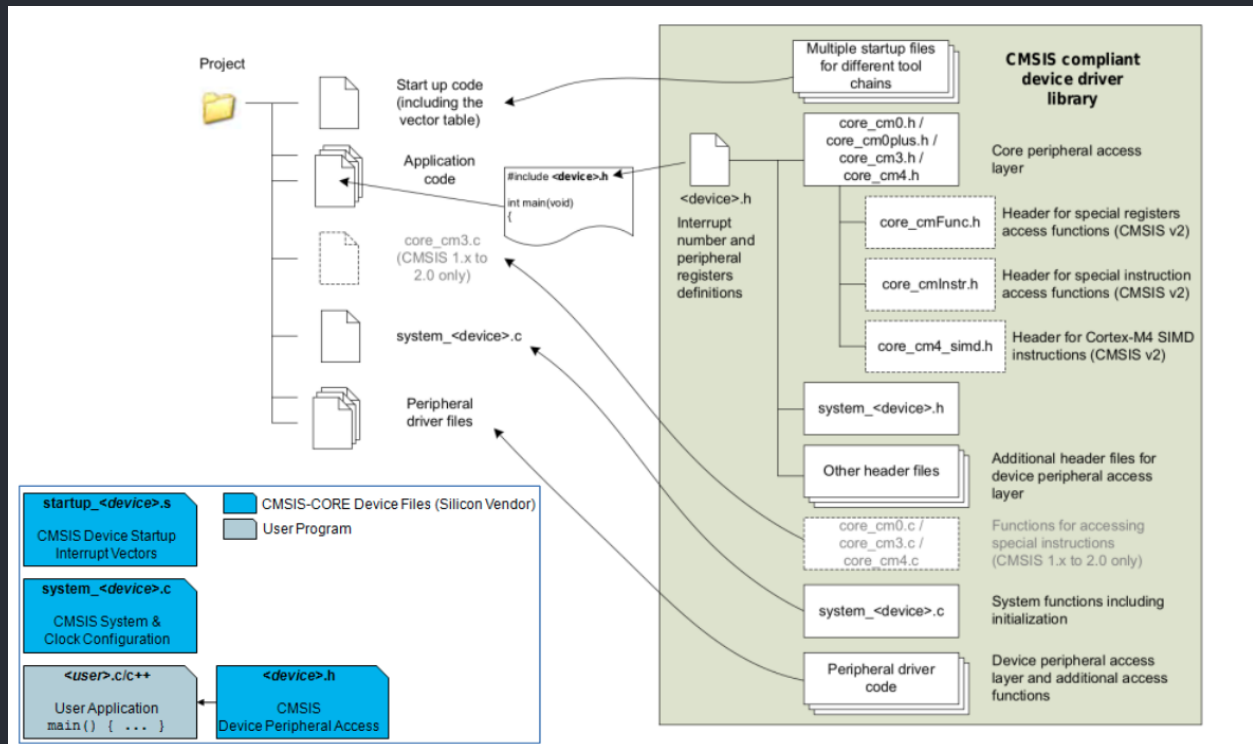
2. SWD method:

برای پروگرام کردن در این روش باید پین های SWDIO, SWCLK, SWO, RST, VDD and GND پروگرامر به پین های AF0, AF0, AF0, AF0, AF12 and AF0 به ترتیب وصل شوند. این پین ها باید به حالت Alternate Function ست شوند.

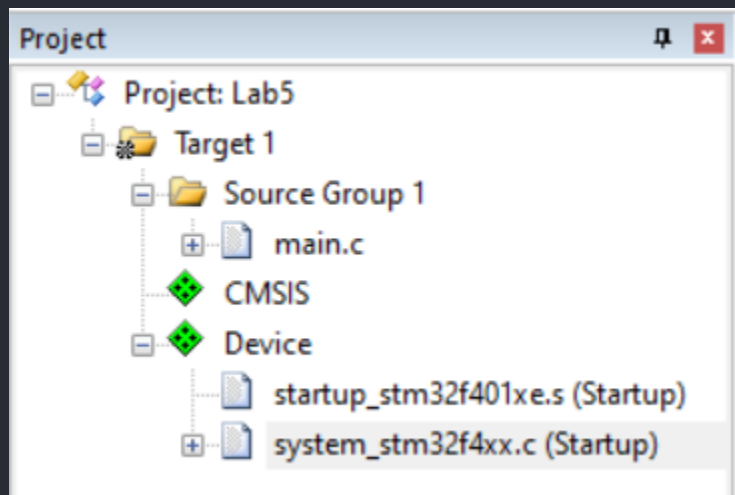
گزارش دستور کار:

توضیح بخش ۱:

شمای کلی فایل های پروژه بر پایه CMSIS:



فایل های ایجاد شده در پروژه به این ترتیب هستند:



توضیحات:

- **startup_stm32f401xe.s (Startup):**

در این فایل Vector Table های اولیه که برای اجرا نیاز است، تعریف می‌شود.

برای مثال استک، هیپ و یکسری هندلر (مثل Rest_Handler، Hardfault_Handler و...) در این فایل استارت‌آپ تعریف می‌شوند:

```
; Vector Table Mapped to Address 0 at Reset
AREA RESET, DATA, READONLY
EXPORT __Vectors
EXPORT __Vectors_End
EXPORT __Vectors_Size

__Vectors DCD __initial_sp          ; Top of Stack
          DCD Reset_Handler        ; Reset Handler
          DCD NMI_Handler          ; NMI Handler
          DCD HardFault_Handler    ; Hard Fault Handler
          DCD MemManage_Handler    ; MPU Fault Handler
          DCD BusFault_Handler     ; Bus Fault Handler
          DCD UsageFault_Handler   ; Usage Fault Handler
          DCD 0                    ; Reserved
          DCD 0                    ; Reserved
          DCD 0                    ; Reserved
          DCD 0                    ; Reserved
          DCD SVC_Handler          ; SVC Call Handler
          DCD DebugMon_Handler     ; Debug Monitor Handler
          DCD 0                    ; Reserved
          DCD PendSV_Handler       ; PendSV Handler
          DCD SysTick_Handler      ; SysTick Handler

; External Interrupts
DCD WWDG_IRQHandler              ; Window WatchDog
DCD PVD_IRQHandler              ; PVD through EXTI Line detection
DCD TAMPER_IRQHandler           ; Tamper and TimeStamps through the EXTI line
DCD RTC_WKUP_IRQHandler        ; RTC Wakeup through the EXTI line
DCD FLASH_IRQHandler            ; FLASH
DCD RCC_IRQHandler              ; RCC
DCD EXTI0_IRQHandler            ; EXTI Line0
DCD EXTI1_IRQHandler            ; EXTI Line1
DCD EXTI2_IRQHandler            ; EXTI Line2
DCD EXTI3_IRQHandler            ; EXTI Line3
DCD EXTI4_IRQHandler            ; EXTI Line4
DCD DMA1_Stream0_IRQHandler     ; DMA1 Stream 0
DCD DMA1_Stream1_IRQHandler     ; DMA1 Stream 1
```

- **system_stm32f401xe.c (Startup):**

این فایل دو تابع و یک متغیر گلوبال برای استفاده در برنامه کاربر تعریف می‌شود، که به شرح زیر است:

```
@brief CMSIS Cortex-M4 Device Peripheral Access Layer System Source File.

This file provides two functions and one global variable to be called from
user application:
- SystemInit(): This function is called at startup just after reset and
before branch to main program. This call is made inside
the "startup_stm32f4xx.s" file.

- SystemCoreClock variable: Contains the core clock (HCLK), it can be used
by the user application to setup the SysTick
timer or configure other parameters.

- SystemCoreClockUpdate(): Updates the variable SystemCoreClock and must
be called whenever the core clock is changed
during program execution.
```

- **SystemInit():** این تابع در استارتاپ و دقیقا بعد از ریست شدن و قبل از برنج شدن و اجرا شدن برنامه اصلی (در اینجا main.c) اجرا می‌شود. این تابع فایل startup ای که بالاتر توضیح داده شد را کال می‌کند.
- **SystemCoreClock variable:** این متغیر برای تنظیم کردن یک سری پارامتر های CoreClock مثل سرعت آن استفاده می‌شود.
- **SystemCoreClockUpdate():** متغیر SystemCoreClock را بروز می‌کند و باید هر جا که این متغیر دچار تغییر شد، صدا زده شود.

کد:

```

1  #include "GPIO.h"
2  #include "stm32f401xe.h"
3
4  static int COEFS[10];
5  static const unsigned char sevenSegHex[10] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
6  static volatile int button_counter = 0;
7  static int N;
8  static int M;
9
10 int pascal(int row, int col);
11 void KHPA(int n);
12 int KHEXT(int n, int m);
13
14 unsigned char bin_to_bcd(int bin);
15
16
17 int main(void) {
18     // Init SevenSegment
19     GPIO_EnableClock(A);
20     for (char i = 0; i < 14; i++) {
21         GPIO_Init(A, i, OUTPUT);
22     }
23
24     // Init DipSwitch
25     GPIO_EnableClock(B);
26     for (char i = 0; i < 4; i++) {
27         GPIO_Init(B, i, INPUT);
28     }
29
30     // Init UserButton
31     GPIO_EnableClock(C);
32     GPIO_Init(C, 0, INPUT);
33
34     // Clear SevenSeg Display
35     for (char i = 0; i < 7; i++) {
36         GPIO_WritePin(A, i, (sevenSegHex[0] >> i) & 0x01);
37     }
38     for (char i = 0; i < 7; i++) {
39         GPIO_WritePin(A, i+7, (sevenSegHex[0] >> i) & 0x01);
40     }

```

```

40 }
41
42 // Init UserButton
43 GPIO_EnableClock(C);
44 GPIO_Init(C, 13, INPUT);
45
46 // Clear SevenSeg Display
47 for (char i = 0; i < 7; i++) {
48     GPIO_WritePin(A, i, (sevenSegHex[0] >> i) & 0x01);
49 }
50 for (char i = 0; i < 7; i++) {
51     GPIO_WritePin(A, i+7, (sevenSegHex[0] >> i) & 0x01);
52 }
53
54 while(1) {
55     if ((GPIO_ReadPin(C, 13) == 0x01) && button_counter == 0) {
56         N = 0;
57         for(char i = 0; i < 4; i++) {
58             N |= (GPIO_ReadPin(B, i) << i);
59         }
60
61         for (char i = 0; i < 7; i++) {
62             GPIO_WritePin(A, i, (sevenSegHex[bin_to_bcd(N)] >> i) & 0x01);
63         }
64
65         button_counter++;
66     } else if ((GPIO_ReadPin(C, 13) == 0x01) && button_counter == 1) {
67         M = 0;
68         for(char i = 0; i < 4; i++) {
69             M |= (GPIO_ReadPin(B, i) << i);
70         }
71
72         for (char i = 0; i < 7; i++) {
73             GPIO_WritePin(A, i+7, (sevenSegHex[bin_to_bcd(M)] >> i) & 0x01);
74         }
75
76         button_counter++;
77     } else if ((GPIO_ReadPin(C, 13) == 0x01) && button_counter == 2) {
78         char bcd = bin_to_bcd(KHEXT(N, M));
79     }

```

```

GPIO.c  GPIO.h  Main.c
81     int digit_1 = bcd & 0x0F;
82     int digit_2 = (bcd >> 4) & 0x0F;
83
84     for (char i = 0; i < 7; i++) {
85         GPIO_WritePin(A, i, (sevenSegHex[digit_1] >> i) & 0x01);
86     }
87     for (char i = 0; i < 7; i++) {
88         GPIO_WritePin(A, i+7, (sevenSegHex[digit_2] >> i) & 0x01);
89     }
90
91     button_counter = 0;
92 }
93 //delay
94 for (int j = 0; j < 1000000; j++) {}
95 }
96 }
97
98 int pascal(int row, int col) {
99     if (col == 1 || row == col)
100         return 1;
101     else
102         return pascal(row - 1, col) + pascal(row - 1, col - 1);
103 }
104
105 void KHPA(int n) {
106     for (int i = 1; i <= n; i++) {
107         COEFS[i] = pascal(n, i);
108     }
109 }
110
111 int KHEXT(int n, int m) {
112     KHPA(n);
113     return COEFS[m];
114 }
115
116 int KHEXT(int n, int m) {
117     KHPA(n);
118     return COEFS[m];
119 }
120
121 unsigned char bin_to_bcd(int bin) {
122     unsigned char bcd = 0x00;
123     int bin_temp = bin;
124
125     for (int i = 0; i < 2; i++) {
126         bcd |= (bin_temp % 10 << 4*(i));
127         bin_temp /= 10;
128     }
129
130     return bcd;
131 }

```